

本章介绍的模块提供了广泛的字符串操作和其他文本处理服务。

在二进制数据服务之下描述的 `codecs` 模块也与文本处理高度相关。此外也请参阅 Python 内置字符串类型的文档文本序列类型 `--- str`。

6.1 string --- 常见的字符串操作

源代码: [Lib/string.py](#)

参见

文本序列类型 `--- str`
字符串的方法

6.1.1 字符串常量

此模块中定义的常量为：

`string.ascii_letters`

下文所述 `ascii_lowercase` 和 `ascii_uppercase` 常量的拼连。该值不依赖于语言区域。

`string.ascii_lowercase`

小写字母 `'abcdefghijklmnopqrstuvwxyz'`。该值不依赖于语言区域，不会发生改变。

`string.ascii_uppercase`

大写字母 `'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`。该值不依赖于语言区域，不会发生改变。

`string.digits`

字符串 `'0123456789'`。

`string.hexdigits`

字符串 `'0123456789abcdefABCDEF'`。

`string.octdigits`

字符串 `'01234567'`。

`string.punctuation`

由在 C 区域设置中被视为标点符号的 ASCII 字符所组成的字符串: `!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~`。

`string.printable`

由被视为可打印符号的 ASCII 字符组成的字符串。这是 `digits`, `ascii_letters`, `punctuation` 和 `whitespace` 的总和。

❗ 备注

根据设计, `string.printable.isprintable()` 将返回 `False`。特别地, `string.printable` 在 POSIX 中是不可打印的 (参见 `LC_CTYPE`)。

`string.whitespace`

由被视为空白符号的 ASCII 字符组成的字符串。其中包括空格、制表、换行、回车、进纸和纵向制表符。

6.1.2 自定义字符串格式化

内置的字符串类提供了通过使用 **PEP 3101** 所描述的 `format()` 方法进行复杂变量替换和值格式化的能力。`string` 模块中的 `Formatter` 类允许你使用与内置 `format()` 方法相同的实现来创建并定制你自己的字符串格式化行为。

class `string.Formatter`

`Formatter` 类包含下列公有方法:

format (`format_string`, `/`, `*args`, `**kwargs`)

首要的 API 方法。它接受一个格式字符串和任意一组位置和关键字参数。它只是一个调用 `vformat()` 的包装器。

在 3.7 版本发生变更: 格式字符串参数现在是 **仅限位置参数**。

vformat (`format_string`, `args`, `kwargs`)

此函数执行实际的格式化操作。它被公开为一个单独的函数, 用于需要传入一个预定义字母作为参数, 而不是使用 `*args` 和 `**kwargs` 语法将字典解包为多个单独参数并重打包的情况。`vformat()` 完成将格式字符串分解为字符数据和替换字段的工作。它会调用下文所述的几种不同方法。

此外, `Formatter` 还定义了一些旨在被子类替换的方法:

parse (`format_string`)

循环遍历 `format_string` 并返回一个由可迭代对象组成的元组 (`literal_text`, `field_name`, `format_spec`, `conversion`)。它会被 `vformat()` 用来将字符串分解为文本字面值或替换字段。

元组中的值在概念上表示一段字面文本加上一个替换字段。如果没有字面文本 (如果连续出现两个替换字段就会发生这种情况), 则 `literal_text` 将是一个长度为零的字符串。如果没有替换字段, 则 `field_name`, `format_spec` 和 `conversion` 的值将为 `None`。

get_field (`field_name`, `args`, `kwargs`)

给定 `field_name` 作为 `parse()` (见上文) 的返回值, 将其转换为要格式化的对象。返回一个元组 (`obj`, `used_key`)。默认版本接受在 **PEP 3101** 所定义形式的字符串, 例如 `"0[name]"` 或 `"label.title"`。`args` 和 `kwargs` 与传给 `vformat()` 的一样。返回值 `used_key` 与 `get_value()` 的 `key` 形参具有相同的含义。

get_value (`key`, `args`, `kwargs`)

提取给定的字段值。`key` 参数将为整数或字符串。如果是整数, 它表示 `args` 中位置参数的索引; 如果是字符串, 它表示 `kwargs` 中的关键字参数名。

`args` 形参会被设为 `vformat()` 的位置参数列表, 而 `kwargs` 形参会被设为由关键字参数组成的字典。

对于复合字段名称, 仅会为字段名称的第一个组件调用这些函数; 后续组件会通过普通属性和索引操作来进行处理。

因此举例来说, 字段表达式 `'0.name'` 将导致调用 `get_value()` 时附带 `key` 参数值 `0`。在 `get_value()` 通过调用内置的 `getattr()` 函数返回后将会查找 `name` 属性。

如果索引或关键字引用了一个不存在的项, 则将引发 `IndexError` 或 `KeyError`。

check_unused_args (*used_args*, *args*, *kwargs*)

在必要时实现对未使用参数进行检测。此函数的参数是格式字符串中实际引用的所有参数的集合 (整数表示位置参数, 字符串表示名称参数), 以及被传给 `vformat` 的 `args` 和 `kwargs` 的引用。未使用参数的集合可以根据这些形参计算出来。如果检测失败则 `check_unused_args()` 应会引发一个异常。

format_field (*value*, *format_spec*)

`format_field()` 会简单地调用内置全局函数 `format()`。提供该方法是为了让子类能够重载它。

convert_field (*value*, *conversion*)

使用给定的转换类型 (来自 `parse()` 方法所返回的元组) 来转换 (由 `get_field()` 所返回的) 值。默认版本支持 `'s'` (`str`), `'r'` (`repr`) 和 `'a'` (`ascii`) 等转换类型。

6.1.3 格式字符串语法

`str.format()` 方法和 `Formatter` 类共享相同的格式字符串语法 (虽然对于 `Formatter` 来说, 其子类可以定义它们自己的格式字符串语法)。具体语法与 格式化字符串面值相似, 但较为简单一些, 并且关键的一点是不支持任意表达式。

格式字符串包含有以花括号 `{}` 括起来的 “替换字段”。不在花括号之内的内容被视为字面文本, 会不加修改地复制到输出中。如果你需要在字面文本中包含花括号字符, 可以通过重复来转义: `{{ and }}`。

替换字段的语法如下:

```
replacement_field ::= "{" [field_name] ["!" conversion] [":" format_spec] "}"
field_name         ::= arg_name ( "." attribute_name | "[" element_index "]" ) *
arg_name           ::= [identifier | digit+]
attribute_name     ::= identifier
element_index      ::= digit+ | index_string
index_string       ::= <any source character except "]"> +
conversion         ::= "r" | "s" | "a"
format_spec        ::= format_spec:format_spec
```

用不太正式的术语来描述, 替换字段开头可以用一个 `field_name` 指定要对值进行格式化并取代替换字符被插入到输出结果的对象。`field_name` 之后有可选的 `conversion` 字段, 它是一个感叹号 `!` 加一个 `format_spec`, 并以一个冒号 `:` 打头。这些指明了替换值的非默认格式。

另请参阅 [格式规格迷你语言](#) 一节。

`field_name` 本身以一个数字或关键字形式的 `arg_name` 打头。如果为数字, 则它指向一个位置参数, 而如果为关键字, 则它指向一个命名关键字参数。如果在字符串上调用 `str.isdecimal()` 会返回真值则 `arg_name` 会被当作数字来处理。如果格式字符串中的数字 `arg_names` 为 `0, 1, 2, ...` 的序列, 它们可以全部 (而非部分) 被省略并且数字 `0, 1, 2, ...` 将按顺序被自动插入。由于 `arg_name` 不使用引号分隔, 因此无法在格式字符串中指定任意的字典键 (例如字符串 `'10'` 或 `':-'` 等)。`arg_name` 之后可以跟任意数量的索引或属性表达式。`'.name'` 形式的表达式会使用 `getattr()` 来选择命名属性, 而 `'[index]'` 形式的表达式会使用 `__getitem__()` 来执行索引查找。

在 3.1 版本发生变更: 位置参数说明符对于 `str.format()` 可以省略, 因此 `'{ } {}'.format(a, b)` 等价于 `'{0} {1}'.format(a, b)`。

在 3.4 版本发生变更: 位置参数说明符对于 `Formatter` 可以省略。

一些简单的格式字符串示例

```
"First, thou shalt count to {0}" # 引用第一个位置参数
"Bring me a {}"                  # 隐式引用第一个位置参数
"From {} to {}"                  # 等同于 "From {0} to {1}"
"My quest is {name}"             # 引用关键字参数 'name'
"Weight in tons {0.weight}"      # 第一个位置参数的 'weight' 属性
"Units destroyed: {players[0]}"  # 关键字参数 'players' 的第一个元素。
```

conversion 字段会在格式化之前进行类型强制转换。通常，格式化一个值的工作是由该值本身的 `__format__()` 方法完成的。但是，在某些情况下最好是强制将类型格式化为一个字符串，覆盖其本身的格式化定义。通过在调用 `__format__()` 之间将值转换为字符串，可以绕过正常的格式化逻辑。

目前支持的转换旗标有三种: `'!s'` 会对值调用 `str()`, `'!r'` 调用 `repr()` 而 `'!a'` 则调用 `ascii()`。

示例如下:

```
"Harold's a clever {0!s}"        # 先在参数上调用 str()
"Bring out the holy {name!r}"    # 先在参数上调用 repr()
"More {!a}"                     # 先在参数上调用 ascii()
```

format_spec 字段包含值应如何呈现的规格描述，例如字段宽度、对齐、填充、小数精度等细节信息。每种值类型可以定义自己的“格式化迷你语言”或对 *format_spec* 的解读方式。

大多数内置类型都支持同样的格式化迷你语言，具体描述见下一节。

format_spec 字段还可以在其内部包含嵌套的替换字段。这些嵌套的替换字段可能包括字段名称、转换旗标和格式规格描述，但是不再允许更深层的嵌套。*format_spec* 内部的替换字段会在解读 *format_spec* 字符串之前先被解读。这将允许动态地指定特定值的格式。

请参阅格式示例一节查看相关示例。

格式规格迷你语言

“格式规格”在格式字符串所包含的替换字段内部使用，用于定义单个值应如何呈现(参见格式字符串语法和 f-strings)。它们也可以被直接传给内置的 `format()` 函数。每种可格式化的类型都可以自行定义如何对格式规格进行解读。

大多数内置类型都为格式规格实现了下列选项，不过某些格式化选项只被数值类型所支持。

一般约定空的格式描述将产生与在值上调用 `str()` 相同的结果。非空格式描述通常会修改此结果。

标准格式说明符的一般形式如下:

```
format_spec ::= [options][width][grouping][." precision][type]
options      ::= [[fill]align][sign]["z"]["#"]["0"]
fill         ::= <any character>
align        ::= "<" | ">" | "=" | "^"
sign         ::= "+" | "-" | " "
width        ::= digit+
grouping     ::= "," | "_"
precision   ::= digit+
type         ::= "b" | "c" | "d" | "e" | "E" | "f" | "F" | "g"
              | "G" | "n" | "o" | "s" | "x" | "X" | "%"
              | "N" | "P" | "Q" | "R" | "S" | "T" | "V" | "W" | "Y" | "Z" | "n" | "N" | "P" | "Q" | "R" | "S" | "T" | "V" | "W" | "Y" | "Z"
```

如果指定了一个有效的 *align* 值，则可以在该值前面加一个 *fill* 字符，它可以为任意字符，如果省略则默认为空格符。在格式化字符串面值或使用 `str.format()` 方法时是无法使用花括号面值 (“{” or “}”) 作为 *fill* 字符的。但是，通过嵌套替换字段插入花括号则是可以的。这个限制不会影响 `format()` 函数。

各种对齐选项的含义如下:

选项	含意
'<'	强制字段在可用空间内左对齐（这是大多数对象的默认值）。
'>'	强制字段在可用空间内右对齐（这是数字的默认值）。
'='	强制在符号（如果有）之后数字之前放置填充。这被用于以'+000000120'形式打印字段。这个对齐选项仅适用于数字类型， <code>complex</code> 除外。当'0'紧接在字段宽度之前时这是默认行为。
'^'	强制字段在可用空间内居中。

请注意，除非定义了最小字段宽度，否则字段宽度将始终与填充它的数据大小相同，因此在这种情况下，对齐选项没有意义。

`sign` 选项仅对数字类型有效，可以是以下之一：

选项	含意
'+'	表示正负号应当同时用于正数与负数。
'-'	表示正负号应当仅用于负数（这是默认的行为）。
space	表示应当对正数使用前导空格，而对负数使用负号。

The `'z'` option coerces negative zero floating-point values to positive zero after rounding to the format precision. This option is only valid for floating-point presentation types.

在 3.11 版本发生变更：增加了 `'z'` 选项（另请参阅 [PEP 682](#)）。

`'#'` 选项可让“替代形式”被用于执行转换。替代形式会针对不同的类型分别定义。此选项仅适用于整数、浮点数和复数类型。对于整数类型，当使用二进制、八进制或十六进制输出时，此选项会为输出值分别添加相应的 `'0b'`、`'0o'`、`'0x'` 或 `'0X'` 前缀。对于浮点数和复数类型，替代形式会使得转换结果总是包含小数点符号，即使其不带小数部分。通常只有在带有小数部分的情况下，此类转换的结果中才会出现小数点符号。此外，对于 `'g'` 和 `'G'` 转换，末尾的零不会从结果中被移除。

`width` 是一个定义最小总字段宽度的十进制整数，包括任何任何前缀、分隔符和其他格式化字符。如果未指定，则字段宽度将由内容确定。

当未显式给出对齐方式时，在 `width` 字符前加一个零（`'0'`）字符将为数字类型启用感知正负号的零填充，`complex` 除外。这相当于将 `fill` 字符设为 `'0'` 并将 `alignment` 类型设为 `'='`。

在 3.10 版本发生变更：在 `width` 字段之前添加 `'0'` 不会再影响字符串的默认对齐。

在 `width` 字段之后的 `grouping` 选项指定用于数字的整数部分的数位组分隔符。它可以为下列值之一：

选项	含意
' , '	对于整数表示类型 <code>'d'</code> 和 <code>'n'</code> 以外的浮点数表示类型每 3 个数位插入一个逗号。对于其他表示类型，此选项不受支持。
' _ '	对于整数表示类型 <code>'d'</code> 和 <code>'n'</code> 以外的浮点数表示类型，每 3 个数位插入一个下划线。对于整数表示类型 <code>'b'</code> 、 <code>'o'</code> 、 <code>'x'</code> 和 <code>'X'</code> ，则每 4 个数位插入一个下划线。对于其他表示类型，此选项不受支持。

想要能感知语言区域的分隔符，请改用 `'n'` 表示类型。

在 3.1 版本发生变更：添加了 `' , '` 选项（另请参阅 [PEP 378](#)）。

在 3.6 版本发生变更：添加了 `' _ '` 选项（另请参阅 [PEP 515](#)）。

`precision` 是一个十进制整数，它表示对于以表示类型 `'f'` 和 `'F'` 格式化的数值应当在小数点后显示多少个数位，或者对于以表示类型 `'g'` 或 `'G'` 格式化的数值应当在小数点前后显示多少个数位。对于字符串表示类型，该字段表示最大的字段大小——换句话说，就是要使用多少个来自字段内容的字符。不允许对整数表示类型指定 `precision` 字段。

最后，`type` 确定了数据应如何呈现。

可用的字符串表示类型是：

类型	含意
's'	字符串格式。这是字符串的默认类型，可以省略。
None	和 's' 一样。

可用的整数表示类型是：

类型	含意
'b'	二进制格式。输出以 2 为基数的数字。
'c'	字符。在打印之前将整数转换为相应的 <code>unicode</code> 字符。
'd'	十进制整数。输出以 10 为基数的数字。
'o'	八进制格式。输出以 8 为基数的数字。
'x'	十六进制格式。输出以 16 为基数的数字，使用小写字母表示 9 以上的数码。
'X'	十六进制格式。输出以 16 为基数的数字，使用大写字母表示 9 以上的数码。在指定 '#' 的情况下，前缀 '0x' 也将被转为大写形式 '0X'。
'n'	数字。这与 'd' 相似，区别在于它会使用当前语言区域设置来插入适当的数字组分隔符。
None	和 'd' 相同。

在上述的表示类型之外，整数还可以通过下列的浮点表示类型来格式化 (除了 'n' 和 `None`)。当这样做时，会在格式化之前使用 `float()` 将整数转换为浮点数。

`float` 和 `Decimal` 值的可用表示类型有：

类型	含意
'e'	科学计数法。对于给定的精度 p ，将数字格式化为以字母'e' 分隔系数和指数的科学计数法表示形式。系数在小数点之前有一位而在之后有 p 位，总计 $p + 1$ 个有效数位。如未指定精度，则会对 <code>float</code> 采用小数点之后 6 位精度，而对 <code>Decimal</code> 则显示所有系数位。如果 $p=0$ ，则小数点会被略去，除非使用了 <code>#</code> 选项。
'E'	科学计数法。与 'e' 相似，不同之处在于它使用大写字母'E' 作为分隔字符。
'f'	定点表示法。对于给定的精度 p ，将数字格式化为小数点之后恰好有 p 位的小数形式。如未指定精度，则会对 <code>float</code> 采用小数点之后 6 位精度，而对 <code>Decimal</code> 则使用大到足够显示所有系数位的精度。如果 $p=0$ ，则小数点会被略去，除非使用了 <code>#</code> 选项。
'F'	定点表示。与 'f' 相似，但会将 <code>nan</code> 转为 <code>NAN</code> 并将 <code>inf</code> 转为 <code>INF</code> 。
'g'	常规格式。对于给定精度 $p \geq 1$ ，这会将数值舍入到 p 个有效数位，再将结果以定点表示法或科学计数法进行格式化，具体取决于其值的大小。精度 0 会被视为等价于精度 1。 准确的规则如下：假设使用表示类型 'e' 和精度 $p-1$ 进行格式化的结果具有指数值 <code>exp</code> 。那么如果 $m \leq \text{exp} < p$ ，其中 m 以 -4 表示浮点值而以 -6 表示 <code>Decimal</code> 值，该数字将使用类型 'f' 和精度 $p-1-\text{exp}$ 进行格式化。否则的话，该数字将使用表示类型 'e' 和精度 $p-1$ 进行格式化。在两种情况下，都会从有效数字中移除无意义的末尾零，如果小数点之后没有余下数字则小数点也会被移除，除非使用了 <code>'#'</code> 选项。如未指定精度，会对 <code>float</code> 采用 6 个有效数位的精度。对于 <code>Decimal</code> ，结果的系数会沿用原值的系数数位；对于绝对值小于 $1e-6$ 的值以及最小有效数位的位值大于 1 的数值将会使用科学计数法，在其他情况下则会使用定点表示法。 正负无穷，正负零和 <code>nan</code> 会分别被格式化为 <code>inf</code> , <code>-inf</code> , <code>0</code> , <code>-0</code> 和 <code>nan</code> ，无论精度如何设定。
'G'	常规格式。类似于 'g'，不同之处在于当数值非常大时会切换为 'E'。无穷与 <code>NaN</code> 也会表示为大写形式。
'n'	数字。这与 'g' 相似，区别在于它会使用当前语言区域来为数字的整数部分插入适当的数字组分隔符。
'%'	百分比。将数字乘以 100 并显示为定点 ('f') 格式，后面带一个百分号。
None	对于 <code>float</code> 来说这类似于 'g' 类型，不同之处在于当使用定点表示形式来格式化结果时，它将总是包括小数点后至少一位，并在 $\text{exp} \geq p - 1$ 时切换为科学计数法表示形式。当未指定精度时，后者的精度将大到足以精确表示给定的值。 对于 <code>Decimal</code> 来说这相当于 'g' 或 'G'，具体取决于当前 <code>decimal</code> 上下文的 <code>context.capitals</code> 值。 总体效果是将 <code>str()</code> 的输出匹配为其他格式化因子所调整出的样子。

结果应当被正确地舍入到给定的小数点后 p 位精度。针对 `float` 的舍入模式与 `round()` 内置函数的相匹配。对于 `Decimal`，将使用当前上下文的舍入模式。

针对 `complex` 的可用表示类型与针对 `float` 的相同（但 '%' 不被允许）。复数的实部和虚部都将被作为浮点数来格式化，使用指定的表示类型。它们将以必须带有的虚部正负符号来分隔，虚部还将带有 `j` 后缀。如果未提供表示类型，结果将匹配 `str()` 的输出（具有非零实部的复数还将带有圆括号），并可能被其他格式说明符所修改。

格式示例

本节包含 `str.format()` 语法的示例以及旧式 % 格式化的比较。

该语法在大多数情况下与旧式的 % 格式化类似，只是增加了 `{}` 和 `:` 来取代 %。例如，`'%03.2f'` 可以被改写为 `'{:03.2f}'`。

新的格式语法还支持新增的不同选项，将在以下示例中说明。

按位置访问参数：

```
>>> '{0}, {1}, {2}'.format('a', 'b', 'c')
'a, b, c'
>>> '{}, {}, {}'.format('a', 'b', 'c') # 3.1+ only
```

(续下页)

(接上页)

```
'a, b, c'
>>> '{2}, {1}, {0}'.format('a', 'b', 'c')
'c, b, a'
>>> '{2}, {1}, {0}'.format(*'abc')      # 解包参数序列
'c, b, a'
>>> '{0}{1}{0}'.format('abra', 'cad')  # 参数的索引可重复使用
'abracadabra'
```

按名称访问参数:

```
>>> 'Coordinates: {latitude}, {longitude}'.format(latitude='37.24N', longitude='-115.81W')
'Coordinates: 37.24N, -115.81W'
>>> coord = {'latitude': '37.24N', 'longitude': '-115.81W'}
>>> 'Coordinates: {latitude}, {longitude}'.format(**coord)
'Coordinates: 37.24N, -115.81W'
```

访问参数的属性:

```
>>> c = 3-5j
>>> ('The complex number {0} is formed from the real part {0.real} '
... 'and the imaginary part {0.imag}.').format(c)
'The complex number (3-5j) is formed from the real part 3.0 and the imaginary part -5.0.'
>>> class Point:
...     def __init__(self, x, y):
...         self.x, self.y = x, y
...     def __str__(self):
...         return 'Point({self.x}, {self.y})'.format(self=self)
...
>>> str(Point(4, 2))
'Point(4, 2)'
```

访问参数的项:

```
>>> coord = (3, 5)
>>> 'X: {0[0]}; Y: {0[1]}'.format(coord)
'X: 3; Y: 5'
```

替代 %s 和 %r:

```
>>> "repr() shows quotes: {!r}; str() doesn't: {!s}".format('test1', 'test2')
'repr() shows quotes: \'test1\'; str() doesn\'t: test2'
```

对齐文本以及指定宽度:

```
>>> '{:<30}'.format('left aligned')
'left aligned'
>>> '{:>30}'.format('right aligned')
'right aligned'
>>> '{:^30}'.format('centered')
'centered'
>>> '{:*^30}'.format('centered')  # 使用 '*' 作为填充字符
'*****centered*****'
```

替代 %+f, %-f 和 % f 以及指定正负号:

```
>>> '{:+f}; {:+f}'.format(3.14, -3.14)  # 总是显示
'+3.140000; -3.140000'
>>> '{: f}; {: f}'.format(3.14, -3.14)  # 对正数显示一个空格
' 3.140000; -3.140000'
>>> '{:-f}; {:-f}'.format(3.14, -3.14)  # 只显示负号 -- 等同于 '{:f}; {:f}'
'3.140000; -3.140000'
```

替代 %x 和 %o 以及转换基于不同进位制的值:

```
>>> # 格式也支持二进制数
>>> "int: {0:d}; hex: {0:x}; oct: {0:o}; bin: {0:b}".format(42)
'int: 42; hex: 2a; oct: 52; bin: 101010'
>>> # with 0x, 0o, or 0b as prefix:
>>> "int: {0:d}; hex: {0:#x}; oct: {0:#o}; bin: {0:#b}".format(42)
'int: 42; hex: 0x2a; oct: 0o52; bin: 0b101010'
```

使用逗号或下划线作为数字组分隔符:

```
>>> '{:,}'.format(1234567890)
'1,234,567,890'
>>> '{:_}'.format(1234567890)
'1_234_567_890'
>>> '{:_b}'.format(1234567890)
'100_1001_1001_0110_0000_0010_1101_0010'
>>> '{:_x}'.format(1234567890)
'4996_02d2'
```

表示为百分数:

```
>>> points = 19
>>> total = 22
>>> 'Correct answers: {:.2%}'.format(points/total)
'Correct answers: 86.36%'
```

使用特定类型的专属格式化:

```
>>> import datetime
>>> d = datetime.datetime(2010, 7, 4, 12, 15, 58)
>>> '{:%Y-%m-%d %H:%M:%S}'.format(d)
'2010-07-04 12:15:58'
```

嵌套参数以及更复杂的示例:

```
>>> for align, text in zip('<>', ['left', 'center', 'right']):
...     '{0:{fill}{align}16}'.format(text, fill=align, align=align)
...
'left<<<<<<<<<<<<'
'^^^^center^^^^'
'>>>>>>>>>>right'
>>>
>>> octets = [192, 168, 0, 1]
>>> '{:02X}{:02X}{:02X}{:02X}'.format(*octets)
'C0A80001'
>>> int(_, 16)
3232235521
>>>
>>> width = 5
>>> for num in range(5, 12):
...     for base in 'dXob':
...         print('{0:{width}{base}}'.format(num, base=base, width=width), end=' ')
...     print()
...
5      5      5      101
6      6      6      110
7      7      7      111
8      8      10     1000
9      9      11     1001
10     A      12     1010
11     B      13     1011
```

6.1.4 模板字符串

模板字符串提供了由 [PEP 292](#) 所描述的更简便的字符串替换方式。模板字符串的一个主要用例是文本国际化 (i18n)，因为在此情境下，更简单的语法和功能使得文本翻译过程比使用 Python 的其他内置字符串格式化工具更方便。作为基于模板字符串构建以实现 i18n 的库的一个例子，请参看 [flufl.i18n](#) 包。

模板字符串支持基于 `$` 的替换，使用以下规则：

- `$$` 为转义符号；它会被替换为单个的 `$`。
- `$identifier` 为替换占位符，它会匹配一个名为 "identifier" 的映射键。在默认情况下，"identifier" 限制为任意 ASCII 字母数字（包括下划线）组成的字符串，不区分大小写，以下划线或 ASCII 字母开头。在 `$` 字符之后的第一个非标识符字符将表明占位符的终结。
- `${identifier}` 等价于 `$identifier`。当占位符之后紧跟着有效的但又不是占位符一部分的标识符字符时需要使用，例如 `"${noun}ification"`。

在字符串的其他位置出现 `$` 将导致引发 `ValueError`。

`string` 模块提供了实现这些规则的 `Template` 类。 `Template` 有下列方法：

class `string.Template(template)`

该构造器接受一个参数作为模板字符串。

substitute(mapping={}, /, **kws)

执行模板替换，返回一个新字符串。 `mapping` 为任意字典类对象，其中的键将匹配模板中的占位符。或者你也可以提供一组关键字参数，其中的关键字即对应占位符。当同时给出 `mapping` 和 `kws` 并且存在重复时，则以 `kws` 中的占位符为优先。

safe_substitute(mapping={}, /, **kws)

类似于 `substitute()`，不同之处是如果有占位符未在 `mapping` 和 `kws` 中找到，不是引发 `KeyError` 异常，而是将原始占位符不加修改地显示在结果字符串中。另一个与 `substitute()` 的差异是任何在其他情况下出现的 `$` 将简单地返回 `$` 而不是引发 `ValueError`。

此方法被认为“安全”，因为虽然仍有可能发生其他异常，但它总是尝试返回可用的字符串而不是引发一个异常。从另一方面来说， `safe_substitute()` 也可能根本算不上安全，因为它将静默地忽略错误格式的模板，例如包含多余的分隔符、不成对的花括号或不是合法 Python 标识符的占位符等等。

is_valid()

如果模板有会导致 `substitute()` 引发 `ValueError` 的无效占位符则返回假值。

Added in version 3.11.

get_identifiers()

返回模板中有效占位符的列表，按它们首次出现的顺序排列，忽略任何无效标识符。

Added in version 3.11.

`Template` 的实例还提供一个公有数据属性：

template

这是作为构造器的 `template` 参数被传入的对象。一般来说，你不应该修改它，但并不强制要求只读访问。

以下是一个如何使用模版的示例：

```
>>> from string import Template
>>> s = Template('$who likes $what')
>>> s.substitute(who='tim', what='kung pao')
'tim likes kung pao'
>>> d = dict(who='tim')
>>> Template('Give $who $100').substitute(d)
Traceback (most recent call last):
...
```

(续下页)

(接上页)

```

ValueError: Invalid placeholder in string: line 1, col 11
>>> Template('$who likes $what').substitute(d)
Traceback (most recent call last):
...
KeyError: 'what'
>>> Template('$who likes $what').safe_substitute(d)
'tim likes $what'

```

进阶用法：你可以派生 `Template` 的子类来自定义占位符语法、分隔符，或用于解析模板字符串的整个正则表达式。为此目的，你可以重载这些类属性：

- *delimiter* -- 这是用来表示占位符的起始的分隔符的字符串字面值。默认值为 `$`。请注意此参数 不能为正则表达式，因为其实现在必要时对此字符串调用 `re.escape()`。还要注意你不能在创建类之后改变此分隔符（例如在子类的类命名空间中必须设置不同的分隔符）。
- *idpattern* -- 这是用来描述不带花括号的占位符的模式正则表达式。默认值为正则表达式 `(?a:[_a-z][_a-z0-9]*)`。如果给出了此属性并且 *braceidpattern* 为 `None` 则此模式也将作用于带花括号的占位符。

备注

由于默认的 *flags* 为 `re.IGNORECASE`，模式 `[a-z]` 可以匹配某些非 ASCII 字符。因此我们在这里使用了局部旗标 `a`。

在 3.7 版本发生变更： *braceidpattern* 可被用来定义对花括号内部和外部进行区分的模式。

- *braceidpattern* -- 此属性类似于 *idpattern* 但是用来描述带花括号的占位符的模式。默认值 `None` 意味着回退到 *idpattern*（即在花括号内部和外部使用相同的模式）。如果给出此属性，这将允许你为带花括号和不带花括号的占位符定义不同的模式。

Added in version 3.7.

- *flags* -- 将在编译用于识别替换内容的正则表达式被应用的正则表达式旗标。默认值为 `re.IGNORECASE`。请注意 `re.VERBOSE` 总是会被加为旗标，因此自定义的 *idpattern* 必须遵循详细正则表达式的约定。

Added in version 3.2.

作为另一种选项，你可以通过重载类属性 *pattern* 来提供整个正则表达式模式。如果你这样做，该值必须为一个具有四个命名捕获组的正则表达式对象。这些捕获组对应于上面已经给出的规则，以及无效占位符的规则：

- *escaped* -- 这个组匹配转义序列，在默认模式中即 `$$`。
- *named* -- 这个组匹配不带花括号的占位符名称；它不应当包含捕获组中的分隔符。
- *braced* -- 这个组匹配带有花括号的占位符名称；它不应当包含捕获组中的分隔符或者花括号。
- *invalid* -- 这个组匹配任何其他分隔符模式（通常为单个分隔符），并且它应当出现在正则表达式的末尾。

如果模式匹配模板但这些命名分组均不匹配则该类上的方法将引发 `ValueError`。

6.1.5 辅助函数

`string.capwords(s, sep=None)`

使用 `str.split()` 将参数拆分为单词，使用 `str.capitalize()` 将单词转为大写形式，使用 `str.join()` 将大写的单词进行拼接。如果可选的第二个参数 *sep* 被省略或为 `None`，则连续的空白字符会被替换为单个空格符并且开头和末尾的空白字符会被移除，否则 *sep* 会被用来拆分和拼接单词。